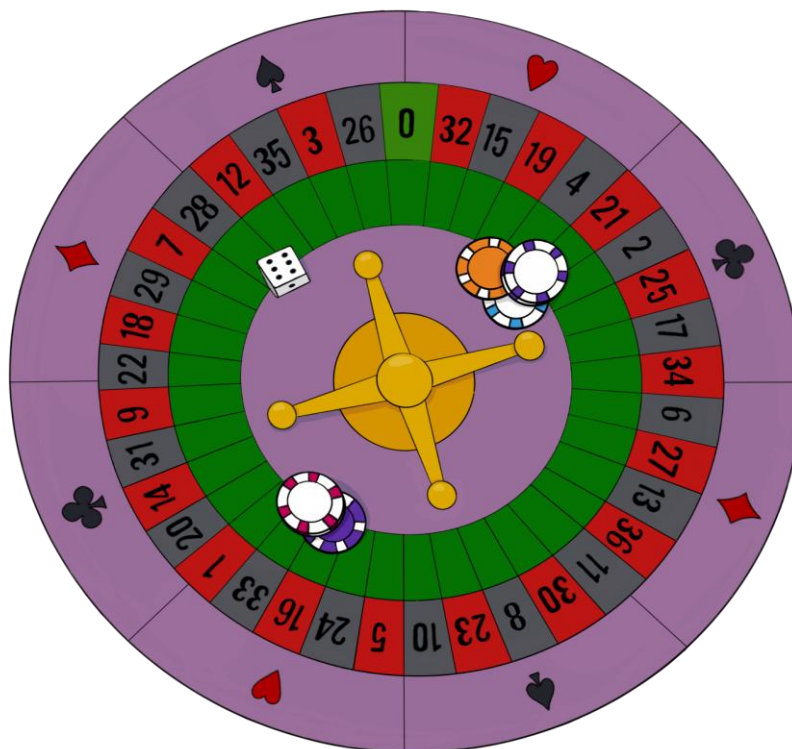


# CASINO



Daniel de las Heras

## Índice

|                                                                |    |
|----------------------------------------------------------------|----|
| 2. Abstract .....                                              | 3  |
| 3. Introducción .....                                          | 4  |
| 3.1 Objetivo .....                                             | 4  |
| 3.2 Justificación .....                                        | 4  |
| 3.3 Análisis de lo existente .....                             | 5  |
| 4. Necesidades empresariales .....                             | 6  |
| Prevención de riesgos laborales .....                          | 6  |
| Forma jurídica .....                                           | 7  |
| Trámites administrativos .....                                 | 7  |
| 5. Requisitos funcionales de la aplicación .....               | 9  |
| 6. Análisis y Diseño .....                                     | 10 |
| 6.1 Diagrama de la arquitectura .....                          | 10 |
| 6.2 Diagrama de casos de uso .....                             | 12 |
| 6.3 Diagrama de clases .....                                   | 13 |
| 6.4 Diseño de datos .....                                      | 14 |
| 7. Codificación .....                                          | 16 |
| 7.1 Entorno de programación .....                              | 16 |
| 7.2 Lenguajes y herramientas .....                             | 17 |
| 7.3 Aspectos relevantes de la implementación .....             | 18 |
| 8. Manual de usuario .....                                     | 26 |
| Uso correcto de la aplicación .....                            | 27 |
| Juego Blackjack Europeo .....                                  | 33 |
| Juego Ruleta Europea .....                                     | 36 |
| 9. Requisitos e instalación .....                              | 37 |
| Requisitos .....                                               | 37 |
| Distribución del APK .....                                     | 37 |
| Procedimiento de instalación .....                             | 37 |
| 10. Conclusión .....                                           | 39 |
| 10.1 Valoración general del proyecto .....                     | 39 |
| 10.2 Dificultades encontradas y soluciones implementadas ..... | 40 |
| 10.3 Funcionalidades pendientes y posibles ampliaciones .....  | 40 |
| 10.4 Futuras mejoras técnicas y de usabilidad .....            | 41 |
| 10.5 Reflexión final .....                                     | 41 |
| 11. Bibliografía .....                                         | 42 |

## 2. Abstract

This project involves the development of a mini-casino application for Android devices, aimed at providing a simple, intuitive, and entertaining user experience through the implementation of classic casino games such as Blackjack, Roulette, and Slots. The main goal is to create an accessible, entertaining solution that serves both as amusement and as a practical demonstration of the knowledge gained during the vocational training .

For its development, Kotlin with Jetpack Compose was used for the user interface, Java with Jakarta EE 10 for building the REST API, and MySQL as the database management system, modeled using Workbench. The architecture follows a client-server pattern, allowing a clear separation between business logic and presentation.

The result is a functional, modular, and easily expandable application that allows users to play, manage their balance, and navigate through different games without complications. Throughout development, challenges related to navigation, interceptors, and JWT (to maintain the user session) were faced, and practical, efficient solutions were implemented.

In conclusion, the project has met its technical and educational goals and is open to future improvements, such as the addition of more games, multiplatform support, and graphical enhancements. The experience has been highly enriching both professionally and academically.

## 3. Introducción

### 3.1 Objetivo

Nuestra empresa desarrollará un producto para comercializar en el mercado: una aplicación móvil centrada en un mini casino, creada completamente desde cero. La app incluirá, en esta primera versión, tres juegos clásicos que suelen encontrarse en casinos reales: la ruleta, el blackjack y una máquina tragaperras. Esta selección responde a un enfoque de tiempo y recursos, permitiendo así concentrarnos en la calidad y funcionalidad básica antes de añadir más contenido.

### 3.2 Justificación

El propósito de este proyecto es brindar una experiencia de casino accesible, sencilla y entretenida para usuarios que busquen diversión sin la complejidad o compromiso económico que implican los casinos tradicionales. A diferencia de otras aplicaciones similares en el mercado, nuestra app ofrecerá inicialmente las mecánicas básicas de los tres juegos mencionados, con planes de incorporar posteriormente recompensas diarias, eventos especiales y nuevas funcionalidades para enriquecer cada modo de juego.

En cuanto a la monetización, la aplicación será gratuita para los usuarios, financiándose mediante anuncios y compras opcionales dentro de la app, garantizando así que nadie esté obligado a pagar para disfrutar del producto.

Desde el punto de vista del desarrollo, el principal reto consiste en programar toda la aplicación desde cero, reduciendo al máximo la dependencia de recursos externos. Cuando sea necesario utilizar activos gráficos, solo se emplearán materiales libres de derechos de autor, asegurando la legalidad y originalidad del producto.

A continuación, se describe brevemente cada uno de los juegos incluidos:

- **Ruleta:** Los jugadores podrán apostar a números, colores o tipos de apuesta especial. La ruleta girará junto con una bola que caerá en un número aleatorio. Si el jugador acierta, recibirá premios acordes a su apuesta.
- **Blackjack:** Un juego de cartas en el que el objetivo es alcanzar 21 puntos o acercarse lo máximo posible sin pasarse. El jugador compite contra la banca, pudiendo decidir si pedir cartas adicionales o plantarse.
- **Máquina tragaperras:** Simulación de las clásicas máquinas tragamonedas donde el usuario inserta créditos y activa el giro. Según la combinación de símbolos obtenidos, el jugador puede ganar diferentes premios.

### 3.3 Análisis de lo existente

Tras un análisis de las aplicaciones y plataformas existentes, se puede afirmar que este proyecto aporta un nivel moderado de innovación. Aunque no es un concepto completamente novedoso — dado que existen numerosas aplicaciones de casino tanto para dispositivos móviles como para web—, no he encontrado proyectos que reúnan en un solo producto las características específicas que planeamos implementar, especialmente en cuanto a la simplicidad inicial combinada con la futura inclusión de recompensas y eventos.

Entre las aplicaciones y páginas web que ofrecen juegos similares destacan plataformas como Playuzu, 888casino, OneCasino o Bet365 en la web; y PlayJango, Genting Casino o Codere en dispositivos móviles. Muchas de estas también incluyen versiones para móviles, lo que demuestra la popularidad del sector.

No obstante, nuestro proyecto se diferencia al enfocarse en una experiencia más accesible, con mecánicas básicas y sin obligar al usuario a realizar pagos, lo que lo hace ideal para un público que busca entretenimiento casual sin compromisos financieros.

## 4. Necesidades empresariales

El propósito de nuestra empresa es ofrecer una oportunidad de aprendizaje a todas aquellas personas interesadas en el desarrollo de aplicaciones móviles Android, programación en Java para API REST, y manejo de bases de datos como MySQL y MongoDB para optimizar consultas. Para ello, se realizarán contratos de prácticas de un mes destinados a quienes quieran adquirir experiencia en estas áreas. Además, se valorará especialmente la incorporación de estudiantes o titulados de grados universitarios en informática o diseño, que posean conocimientos básicos de programación.

En la fase inicial, la plantilla estará compuesta por dos personas: un desarrollador encargado de la programación de la aplicación y un diseñador gráfico responsable tanto de los recursos visuales como del diseño de la interfaz de usuario.

### Prevención de riesgos laborales

Respecto a la prevención de riesgos laborales, es fundamental que los trabajadores mantengan una postura correcta frente al ordenador para evitar molestias y lesiones, especialmente en la espalda y cuello. Se recomienda descansar brevemente cada hora de trabajo continuado y utilizar una silla ergonómica de oficina que proporcione soporte adecuado y comodidad durante la jornada laboral.



### Forma jurídica

La empresa se constituirá bajo la forma jurídica de Sociedad de Responsabilidad Limitada (S.L.). Esta opción es la más común y adecuada para negocios pequeños y nuevos como el nuestro, ya que ofrece múltiples ventajas legales y fiscales en España. Además, el capital social mínimo requerido

para su constitución es de solo 1 euro, lo que facilita la puesta en marcha sin necesidad de grandes inversiones iniciales. La S.L. también proporciona una estructura flexible y sencilla para gestionar la empresa.

|                                       |                                                                                              |
|---------------------------------------|----------------------------------------------------------------------------------------------|
| <b>ECONFINADOS</b>                    | <b>SOCIEDAD LIMITADA (S.L.)</b>                                                              |
| <b>CAPITAL MÍNIMO DE CONSTITUCIÓN</b> | <b>3.000 € suscritos y desembolsados</b><br>(este capital se divide en participaciones)      |
| <b>TRANSMISIÓN DE LA PROPIEDAD</b>    | <b>Libre:</b> entre socios o familiares directos<br><b>Restringida:</b> al resto de personas |
| <b>TIPO DE RESPONSABILIDAD</b>        | Limitada al capital suscrito                                                                 |
| <b>Nº DE SOCIOS</b>                   | Puede ser uno (SL Unipersonal) o varios                                                      |
| <b>TRIBUTACIÓN</b>                    | Impuesto de sociedades (IS)                                                                  |

## Trámites administrativos

Para formalizar la empresa, el primer paso será registrar el nombre comercial en el Registro Mercantil Central, asegurándonos de que no exista otra empresa con el mismo nombre. Una vez registrado, se recibirá un certificado que reserva el nombre durante seis meses, renovable si es necesario.

Posteriormente, se abrirá una cuenta bancaria a nombre de la empresa. Los socios redactarán los Estatutos Sociales, que son las normas que regirán la empresa, y procederán a firmar la escritura pública de constitución de la S.L. ante notario.

A continuación, se realizarán los trámites fiscales en Hacienda, y se inscribirá la empresa en el Registro Mercantil correspondiente a la provincia donde esté establecido el domicilio social. Este trámite debe completarse dentro de los dos meses siguientes a la obtención de la escritura.

Por último, se debe tramitar el cambio de la tarjeta provisional del N.I.F. por la definitiva, registrarse como autónomo societario y obtener el certificado digital de administrador de la sociedad, indispensable para gestionar electrónicamente los asuntos legales y fiscales.

## 5. Requisitos funcionales de la aplicación

El objetivo principal de este proyecto es resolver un problema común para muchas personas: el aburrimiento y la necesidad de juegos casuales para el ocio y entretenimiento. La aplicación está diseñada para ofrecer una experiencia rápida y accesible, ideal para momentos en los que se dispone de poco tiempo, pero se quiere disfrutar de un rato ameno, como mientras se espera o se está en tránsito.

Esta app está dirigida a usuarios mayores de 18 años que buscan una forma sencilla de diversión, estimulante y cargada de la emoción que proporcionan los juegos de azar. Para ello, se ha diseñado una interfaz amigable y fácil de usar, que no requiere conocimientos previos ni experiencia en juegos de casino.

La aplicación será utilizada por personas que quieren entretenerse de manera rápida y sin complicaciones, ya sean jugadores novatos o usuarios ocasionales que disfruten de juegos casuales. En caso de cualquier duda o problema, los usuarios podrán contactar con un servicio de soporte dedicado, dispuesto a resolver consultas y atender sugerencias. Asimismo, se fomentará la recepción de críticas constructivas para seguir mejorando la experiencia y funcionalidad de la app.

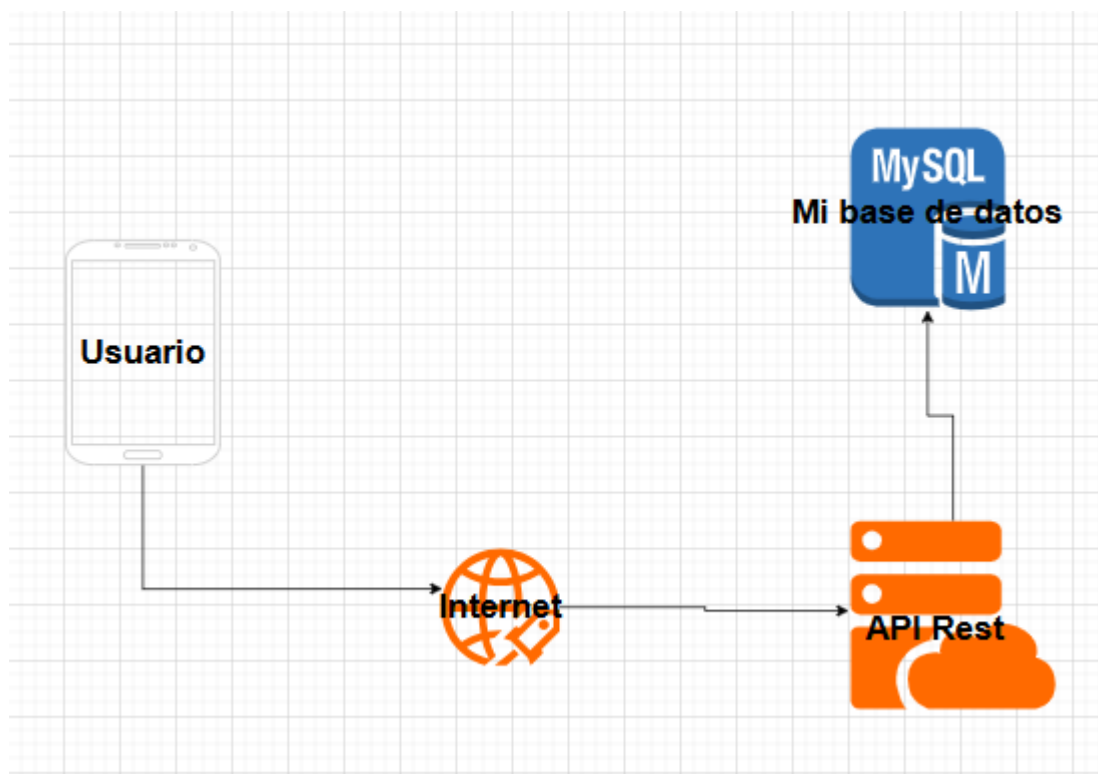


## 6. Análisis y Diseño

### 6.1 Diagrama de la arquitectura

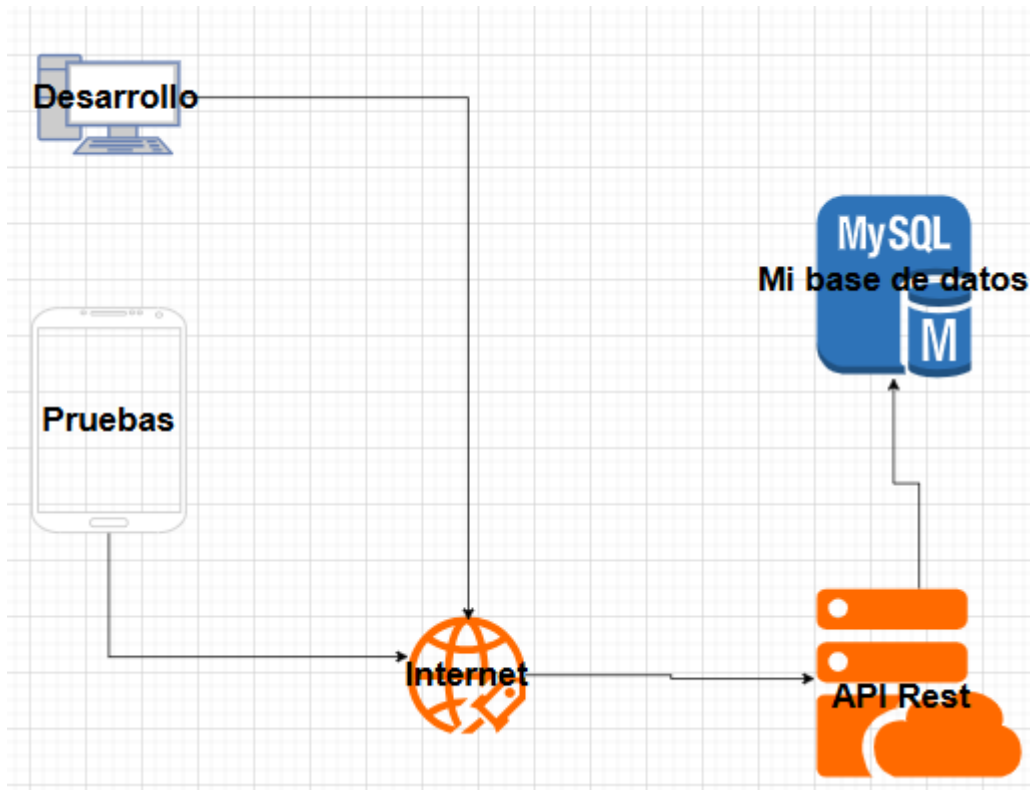
Infraestructura del cliente: Link

La aplicación móvil requiere que el cliente tenga conexión a internet para su correcto funcionamiento. El API REST, encargado de gestionar la lógica y la comunicación con la base de datos, está alojado en un servicio en la nube, Microsoft Azure, lo que permite un acceso rápido y seguro desde cualquier dispositivo. En caso de que el usuario no disponga de conexión, la aplicación mostrará un mensaje de error indicando la imposibilidad de acceder a los servicios.



Infraestructura de la empresa: [Link](#)

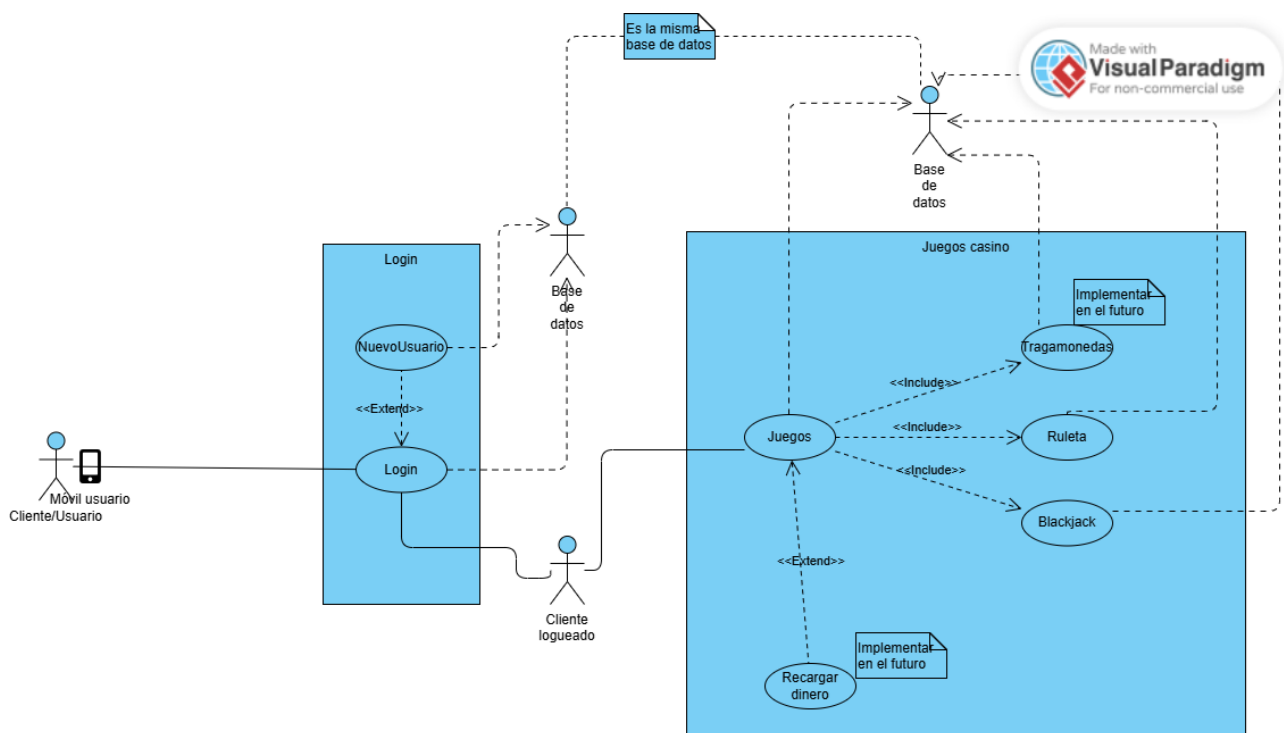
Por otro lado, la infraestructura de la empresa está diseñada para cubrir tanto el desarrollo como las pruebas del sistema durante todo el proceso de creación de la aplicación.



## 6.2 Diagrama de casos de uso

En el diagrama de casos de uso se observa el flujo principal de la aplicación desde la perspectiva del usuario. Al abrir la app, el cliente accederá primero a la pantalla de inicio de sesión. En caso de no tener una cuenta, deberá registrarse para poder continuar. Una vez dentro, podrá seleccionar cualquiera de los juegos disponibles, los cuales están conectados a la misma base de datos.

Por diseño, he considerado dos actores para representar distintas interacciones, aunque en esta primera versión algunos juegos aún no estarán disponibles. Por ejemplo, el juego de máquina tragaperras no se incluirá inicialmente, y la opción de recargar dinero tampoco estará activa. Además, es posible que la ruleta solo esté disponible en forma de prototipo. Debido a limitaciones de tiempo, se contempla realizar una demostración en la defensa del proyecto mostrando cómo funcionaría la ruleta.



## 6.3 Diagrama de clases

Los diagramas UML de la lógica del casino y de las interfaces gráficas están disponibles en los siguientes enlaces:

PlantUML casinoLogica: [Link](#)

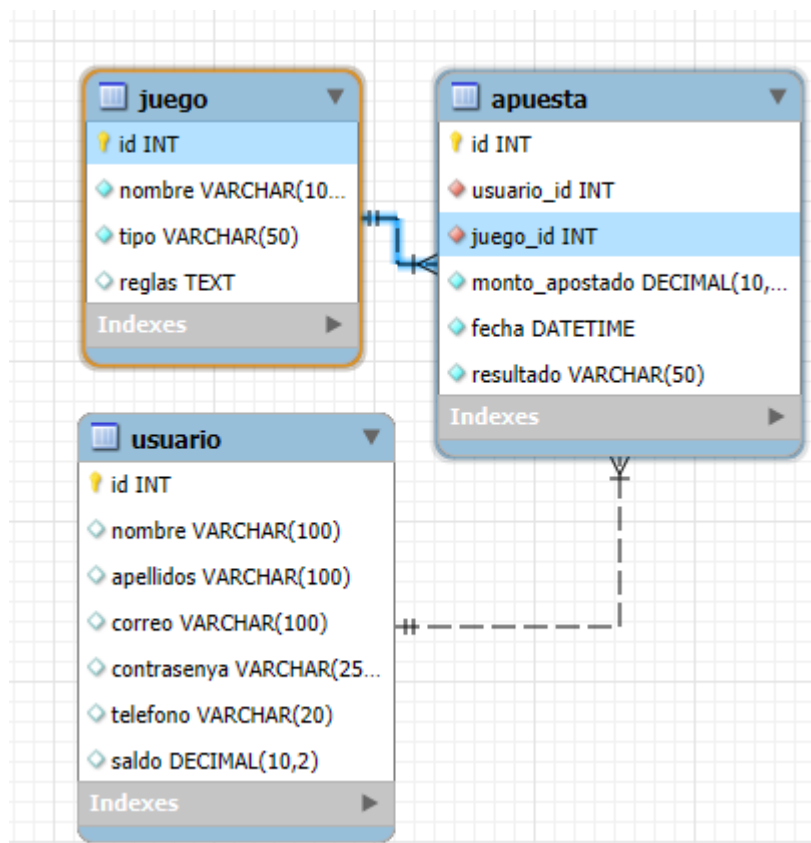
PlantUML casinoScreen: [Link](#)

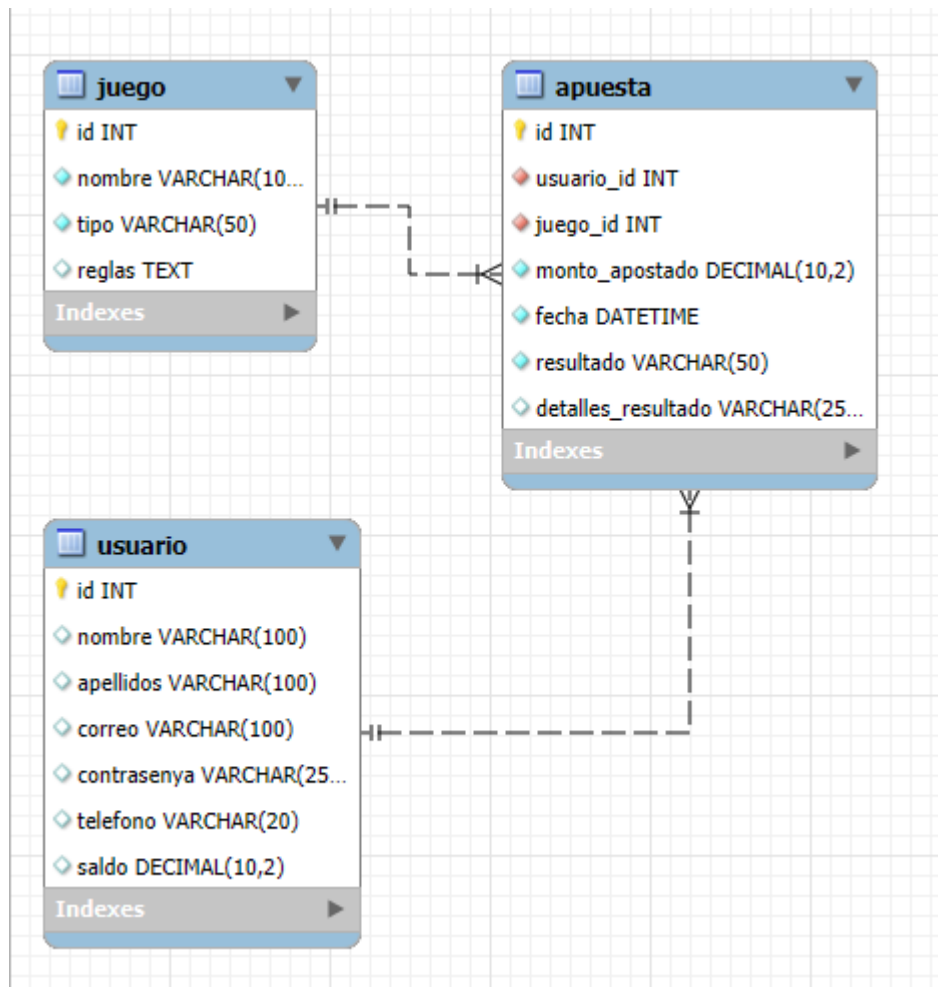
Estos diagramas reflejan en detalle las clases que conforman la aplicación, incluyendo sus atributos y métodos, así como las relaciones entre ellas. Debido a su extensión, no se incluyen imágenes aquí para evitar una presentación desordenada.

## 6.4 Diseño de datos

La base de datos relacional consta de tres tablas principales:

- **Usuario:** almacena la información personal y de acceso de cada usuario registrado.
- **Juego:** contiene los datos relacionados con cada juego disponible en la aplicación.
- **Apuesta:** registra las partidas realizadas, incluyendo la cantidad apostada, el resultado (ganado o perdido) y la relación entre usuario y juego, permitiendo así generar estadísticas sobre los juegos más populares.





Esquema de Entidad/Relación: [Link](#)

Esquema de Entidad/Relación nuevo: [Link](#)

Además, se utiliza MongoDB, una base de datos NoSQL, para gestionar las peticiones de apuestas en tiempo real. Dado que estas operaciones pueden ser numerosas y costosas en términos de rendimiento, MongoDB permite mantener los datos en memoria de forma eficiente. Una vez finalizada cada partida, los registros se transfieren a la base de datos relacional para su almacenamiento definitivo. La estructura de los documentos en MongoDB replica la tabla de apuestas, facilitando así la sincronización y el traspaso de datos entre ambas bases.

Los scripts de creación y modificación de la base de datos pueden consultarse en los siguientes enlaces:

- Script de la base de datos: [Link](#)
- Cambio a la tabla apuestas: [Link](#)

## 7. Codificación

### 7.1 Entorno de programación

Para el desarrollo de la aplicación móvil se ha utilizado **Android Studio**, dado que es el entorno oficial para desarrollo en Android y está especialmente diseñado para trabajar con Kotlin y Java. En este proyecto, la aplicación móvil se ha programado principalmente en Kotlin.

Para el desarrollo del API REST en Java se ha empleado **NetBeans**, un entorno que destaca por su sencillez y bajo consumo de recursos, ideal para desarrollar backend sin complicaciones.

En cuanto a la gestión de la base de datos relacional, se utilizó **MySQL Workbench**. Esta herramienta facilita la creación y gestión de esquemas Entidad-Relación (E/R), además de ser intuitiva y ampliamente usada.

## 7.2 Lenguajes y herramientas

Este proyecto combina varios lenguajes y tecnologías para ajustarse a las necesidades específicas de cada componente:

- **Kotlin con Jetpack Compose:** para la aplicación móvil. Kotlin es un lenguaje moderno, promovido por Google, optimizado para Android y que facilita la creación de interfaces reactivas y fluidas gracias a Jetpack Compose. Además, cuenta con buen soporte y librerías externas que simplifican tareas comunes.
- **Java con Jakarta EE 10:** para el desarrollo del API REST. Jakarta EE ofrece una plataforma robusta y estandarizada para construir servicios web de manera sencilla y eficiente, ideal para aplicaciones servidoras.
- **MySQL:** base de datos relacional para almacenar información estructurada como usuarios, juegos y apuestas. Se eligió por su fiabilidad y facilidad para manejar datos simples y estructurados.
- **MongoDB:** base de datos NoSQL utilizada para almacenar temporalmente las partidas y apuestas en tiempo real. Gracias a su modelo de documentos, es más eficiente para operaciones frecuentes y rápidas, permitiendo una mejor performance antes de sincronizar los datos a la base relacional.



## 7.3 Aspectos relevantes de la implementación

Empezaré explicando los **TypeAdapter**. En Java, esta clase es útil cuando no se quiere usar la clase `Date` porque muchos métodos están deprecados y se recomienda usar `LocalDateTime` u otras clases de la API de tiempo. Por ejemplo:

```
public class LocalDateTimeAdapter extends TypeAdapter<LocalDateTime>{  
  
    @Override  
    public void write(JsonWriter writer, LocalDateTime t) throws IOException {  
        writer.value(value: t.format(formatter: DateTimeFormatter.ISO_LOCAL_DATE_TIME));  
    }  
  
    @Override  
    public LocalDateTime read(JsonReader reader) throws IOException {  
        return LocalDateTime.parse(text: reader.nextString(), formatter: DateTimeFormatter.ISO_LOCAL_DATE_TIME);  
    }  
}
```

Esto es necesario porque la librería Gson, que se usa para serializar objetos, no soporta de forma nativa clases como `LocalDateTime`. Por eso se crea un adaptador personalizado que convierte estos objetos para que Gson los pueda manejar sin errores.

Ahora explicaré qué es **JWT** (JSON Web Token), una librería muy usada para crear tokens que mantienen la sesión abierta en aplicaciones. En mi caso, cuando el usuario inicia sesión puede elegir si mantenerla activa o no. Para ello, en Java he creado una clase sencilla que facilita crear y validar estos tokens:

```
// Clave secreta para firmar el JWT
private static final SecretKey SECRET_KEY = generateSecretKey();

// Generar la clave secreta HMAC-SHA256
private static SecretKey generateSecretKey() {
    try {
        // Crear un generador de claves para HMAC-SHA256
        KeyGenerator keyGenerator = KeyGenerator.getInstance(algorithm: "HmacSHA256");
        keyGenerator.init(keysize:256); // Tamaño de la clave
        return keyGenerator.generateKey();
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(message:"Error generando la clave secreta", cause: e);
    }
}

// Generar el token JWT
public static String generarToken(String username) {
    return Jwts.builder()
        .subject(string: username)
        .issuedAt(date: Date.from(instant:Instant.now()))
        .expiration(date: Date.from(instant:Instant.now().plus(amountToAdd:1, unit: ChronoUnit.MONTHS)))
        .signWith(k: SECRET_KEY, sda:Jwts.SIG.HS256)
        .compact();
}

// Verificar y obtener el subject del token (en este caso, el username)
public static String getCorreoFromToken(String token) {
    return Jwts.parser()
        .verifyWith(sk: SECRET_KEY)
        .build()
        .parseSignedClaims(cs: token)
        .getPayload()
        .getSubject(); // Obtener el correo del token
}
```

En la parte cliente se ha creado otra clase para manejar las funciones básicas del token. Este se guarda en **DataStore**, la nueva API recomendada por Google que reemplaza a SharedPreferences. Algunas diferencias:

- DataStore es moderna, usa Kotlin coroutines y flows, es asíncrona y no bloquea el hilo principal.
- Tiene mejor manejo de concurrencia y es más fácil de testear.
- SharedPreferences es la API clásica, guarda en un archivo XML, es síncrona y puede bloquear el hilo si se usa mucho.

```
// Crea un delegado de extensión para acceder al DataStore en el contexto de la aplicación
val Context.dataStore by preferencesDataStore(name = "user_prefs")

// TokenManager maneja el almacenamiento y recuperación del token JWT en el DataStore
object TokenManager {
    // Clave para almacenar y recuperar el token JWT en el DataStore
    private val JWT_KEY = stringPreferencesKey(name = "jwt_token")

    // Función para guardar el token
    suspend fun saveToken(context: Context, token: String) {
        context.dataStore.edit { preferences ->
            preferences[JWT_KEY] = token
        }
    }

    // Función para obtener el token
    suspend fun getToken(context: Context): String? {
        val preferences = context.dataStore.data.first()
        return preferences[JWT_KEY]
    }

    // Función para borrar el token
    suspend fun clearToken(context: Context) {
        context.dataStore.edit { preferences ->
            preferences.remove(JWT_KEY)
        }
    }
}
```

En esta clase, se crea una carpeta llamada user\_prefs, donde de momento solo se guarda el jwt\_token. Se puede guardar, borrar o leer el token fácilmente.

En cuanto a los **interceptores**:

- En el servidor, hay un interceptor que revisa cada petición al API REST para comprobar si el encabezado HTTP trae el token. Se implementa con ContainerRequestFilter y su método:

```
@Provider
public class AuthFilter implements ContainerRequestFilter {
```

Esta primera parte comprueba si el token está presente, lo separa de la cadena y extrae el correo para validar el usuario. Si no hay token, ocurre esto:

```
@Override
public void filter(ContainerRequestContext requestContext) throws IOException {
    // Obtener el token del header Authorization
    String authorizationHeader = requestContext.getHeaderString(string: HttpHeaders.AUTHORIZATION);

    if (authorizationHeader != null && authorizationHeader.startsWith(prefix: "Bearer ")
        && !authorizationHeader.equals(anObject: "Bearer null")) {
        // Extraer el token
        String token = authorizationHeader.substring(beginIndex: "Bearer".length()).trim();

        try {
            // Validar el token
            String correo = JwtUtil.getCorreoFromToken(token);

            if (correo == null) {
                // Si el token no es válido o ha expirado
                requestContext.abortWith(respns: Response.status(status: Status.UNAUTHORIZED)
                    .entity(o: "Token inválido o expirado")
                    .build());
            } else {
                // Si el token es válido, puedes agregar el usuario al contexto
                requestContext.setProperty(string: "usuarioCorreo", o: correo);
            }
        } catch (Exception e) {
            // Si ocurre un error durante la validación del token
            requestContext.abortWith(respns: Response.status(status: Status.UNAUTHORIZED)
                .entity(o: "Token no válido")
                .build());
        }
    }

    // Si no hay token, continuar la solicitud
    // Puedes permitir la creación de usuarios sin token
    requestContext.setProperty(string: "usuarioCorreo", o: "sin-token");
}
```

Esto permite proteger rutas que requieren autenticación y evitar repetir el código en cada método.

- También existe una clase para manejar excepciones globalmente en el API REST, enviando respuestas con errores sin tener que usar try-catch en cada método.

```
@Provider
public class GenericExceptionHandler implements ExceptionMapper<Exception> {

    private static final System.Logger LOG = System.getLogger(name: GenericExceptionHandler.class.getName());

    @Override
    public Response toResponse(Exception ex) {
        LOG.log(level: System.Logger.Level.ERROR, msg: ex.getMessage());
        return Response
            .status(status: Status.INTERNAL_SERVER_ERROR)
            .build();
    }
}
```

En el cliente Android, se crearon otros interceptores:

- Uno que obtiene el token de DataStore y lo añade al encabezado de cada petición al API REST automáticamente.

```
class AuthInterceptor @Inject constructor(  
    @ApplicationContext private val context: Context  
) : Interceptor {  
  
    override fun intercept(chain: Interceptor.Chain): Response {  
        return try {  
            // Obtener el token desde DataStore  
            val token: String? = runBlocking {  
                withContext(Dispatchers.IO) {  
                    TokenManager.getToken(context)  
                }  
            }  
        }  
    }  
}
```

```
Log.d(  
    tag: "AuthInterceptor",  
    msg: "Token: $token"  
) // Log para verificar si se ejecuta el interceptor  
  
// Si el token es válido, agregarlo al encabezado  
val requestBuilder = chain.request().newBuilder()  
  
token.let {  
    requestBuilder.addHeader(name: "Authorization", value: "Bearer $it")  
}  
  
// Mostrar el encabezado en el log  
Log.d(  
    tag: "Request",  
    msg: "Authorization Header: " + requestBuilder.build().header(name: "Authorization")  
)  
  
// Continuar con la solicitud  
chain.proceed(requestBuilder.build())
```

```
catch (e: SocketTimeoutException) {
    Response.Builder()
        .request(chain.request())
        .protocol(Protocol.HTTP_1_1)
        .message("Socket time out")
        .body("").toResponseBody("application/json".toMediaTypeOrNull())
        .code(598)
        .build()
}
catch (e: ConnectException) {
    Response.Builder()
        .request(chain.request())
        .protocol(Protocol.HTTP_1_1)
        .message("Fail connect")
        .body("").toResponseBody("application/json".toMediaTypeOrNull())
        .code(599)
        .build()
}
```

- Otro interceptor que verifica si el dispositivo está conectado por WiFi o datos móviles, para controlar el acceso a internet.

```
class ConnectVerifierInterceptor @Inject constructor(
    private val networkMonitor: NetworkMonitorService
) : Interceptor {

    override fun intercept(chain: Interceptor.Chain): Response {
        return try {
            val request = chain.request()

            if (networkMonitor.isConnected()) {
                return chain.proceed(request)
            }

            throw NoNetworkException("Network Error")
        } catch (e: NoNetworkException) {
            Response.Builder()
                .request(chain.request())
                .protocol(Protocol.HTTP_1_1)
                .message("No network")
                .body("").toResponseBody("application/json".toMediaTypeOrNull())
                .code(499)
                .build()
        }
    }
}
```

Esta clase fue sencilla de implementar, ya que solo comprueba el estado de conexión.

```
class NetworkMonitorServiceImpl @Inject constructor(  
    @ApplicationContext private val context: Context  
) : NetworkMonitorService {  
    private val connectivityManager =  
        context.getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager  
  
    override fun isConnected(): Boolean {  
        return connectivityManager.activeNetwork != null  
    }  
}
```

Finalmente, explicaré uno de los errores más difíciles de solucionar. Cuando el API REST estaba apagado o no respondía, la app móvil se cerraba sin aviso porque bloqueaba el hilo principal. Después de mucho depurar, detecté que el error no se capturaba en los interceptores ni en los ViewModels, y por eso la app colapsaba.

Para solucionarlo, añadí bloques try-catch y modifiqué el manejo de errores para que el interceptor envíe la respuesta con el error al ViewModel, que puede procesarlo y mostrar mensajes adecuados sin cerrar la app. Ejemplo:

```
suspend fun getCarta(): CartaApi? {  
    val mensajeError = "No se ha podido obtener la carta"  
  
    try {  
        val response = blackJackService.getCarta()  
        validarCodigoResponse(response)  
    }  
}
```

```
fun <T> validarCodigoResponse(response: Response<T>) {  
    when (response.code()) {  
        499 -> throw NoNetworkException("No hay red")  
        598 -> throw SocketTimeoutException("Finalizado tiempo espera")  
        599 -> throw ConnectException("Error al conectar")  
    }  
}
```



## 8. Manual de usuario

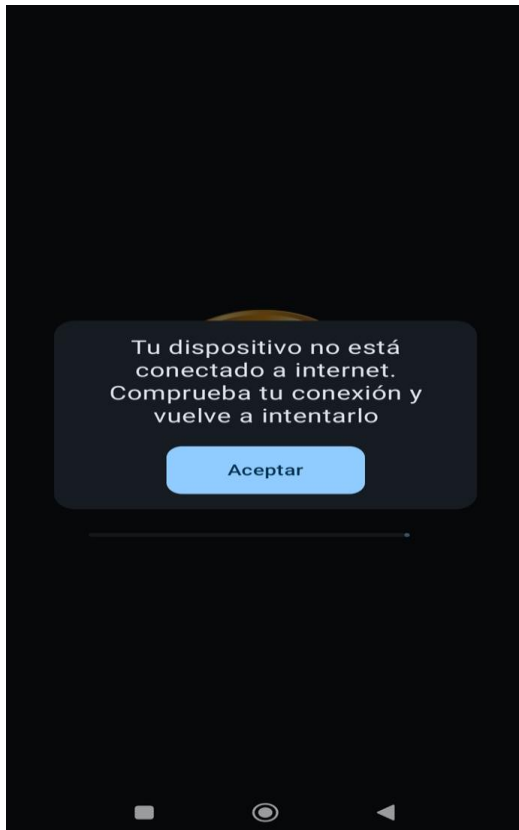
Esta aplicación está destinada a personas mayores de 18 años, que estén en sus plenas facultades. Principalmente es una aplicación para entretenimiento y ocio.

Antes de empezar, recomendamos no usar el móvil más de **1 hora seguida** por razones de salud, ya que un uso prolongado puede afectar la vista y causar molestias musculares. También es importante mantener una buena postura para evitar problemas en la espalda o cuello.

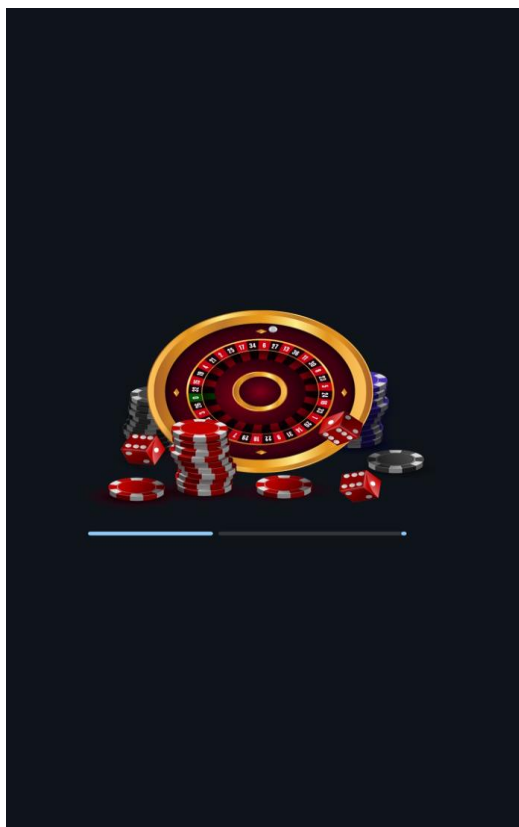


## Uso correcto de la aplicación

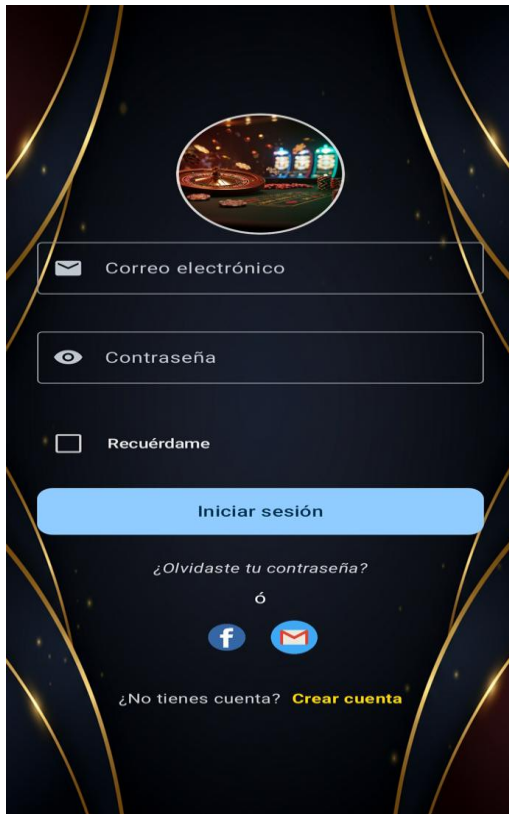
- Si el dispositivo no tiene conexión a internet o el API REST está apagado, se mostrará un mensaje de error indicando la falta de conexión.

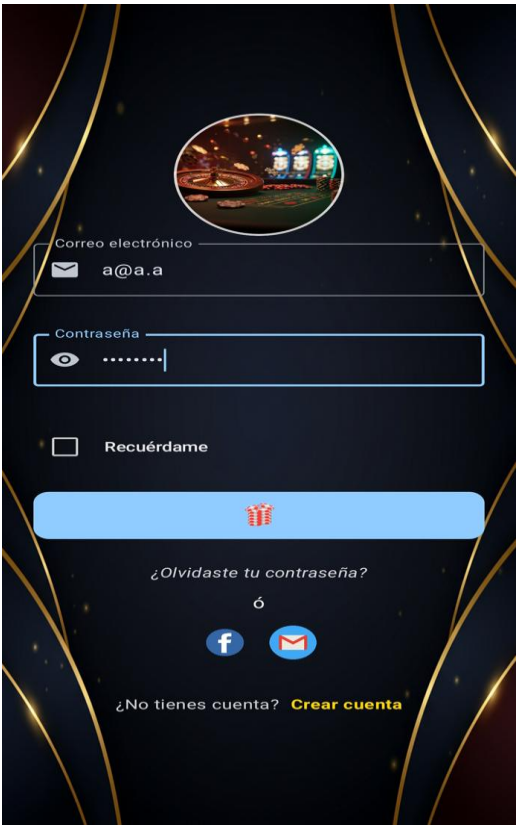


- Cuando todo funcione correctamente, aparecerá una pantalla de carga que comprobará si el usuario tiene la sesión iniciada o no.

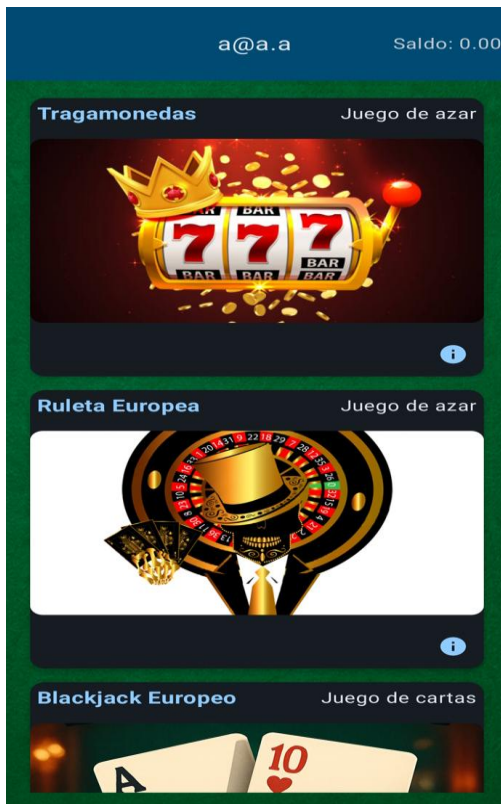


- Al entrar en la aplicación, aparecerá la pantalla de **login**, donde podrás acceder con tu cuenta. Si no tienes una, puedes crear un usuario nuevo pulsando el botón amarillo.





- Después de iniciar sesión, aparecerán los tres juegos disponibles: **Blackjack Europeo**, **Ruleta Europea** y **Tragamonedas**.

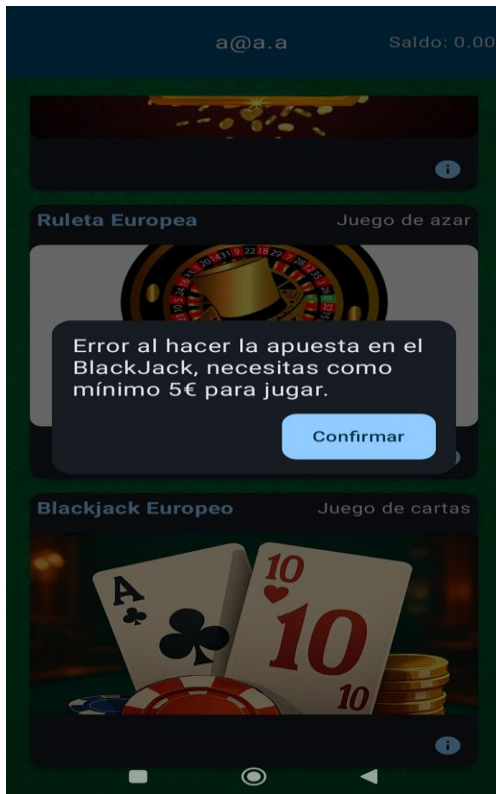


- Si no conoces bien algún juego, el icono azul te mostrará una explicación para ayudarte a entender las reglas.



## Juego Blackjack Europeo

- La apuesta mínima es de **5 €**. Si no tienes saldo suficiente, la aplicación mostrará un error.



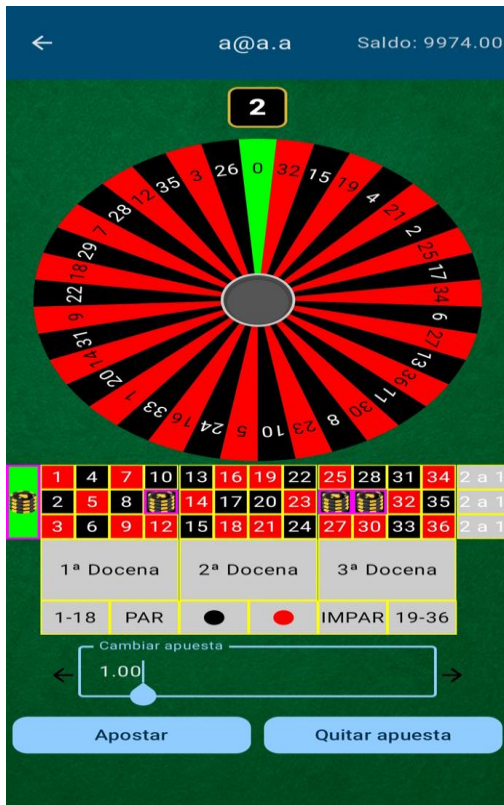


- En el juego, la pantalla está dividida en dos mitades: la parte superior es del **Crupier**, donde se muestran sus cartas, y la inferior es del usuario. En el centro hay una decoración para evitar que la pantalla quede vacía.
- Las acciones del jugador son dos: **Pedir carta** o **Plantarse**. Si la suma de las cartas supera 21, pierdes automáticamente.





## Juego Ruleta Europea



- El juego cuenta con un temporizador que da tiempo para realizar las apuestas.
- La ruleta gira junto a una pelota que, al terminar el temporizador, caerá en un número aleatorio.
- Hay dos botones principales: uno para realizar la apuesta y otro para retirarla.

## 9. Requisitos e instalación

### Requisitos

Para poder usar esta aplicación es necesario contar con un sistema operativo Android versión **9.0** como mínimo. Sin embargo, se recomienda usar la versión **11.0 o superior**, ya que algunas funcionalidades solo están disponibles en estas versiones más recientes y no se han probado en versiones anteriores a la 10.

### Distribución del APK

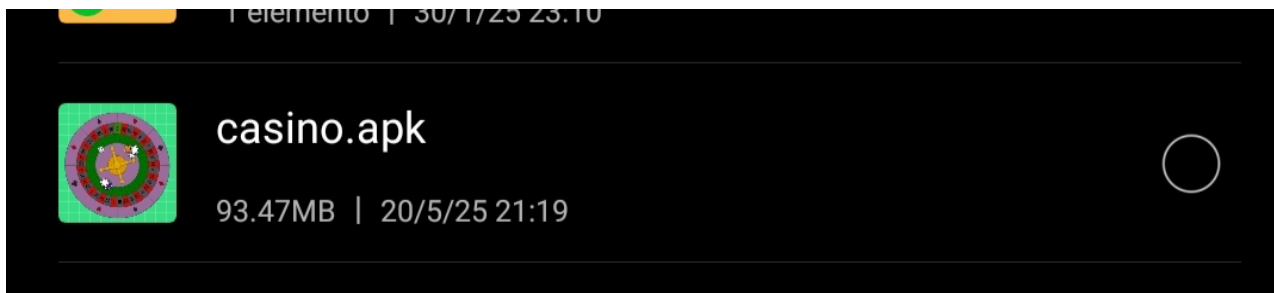
La aplicación se ha desarrollado en Android Studio y se ha generado un archivo APK firmado mediante una clave creada para este fin, lo que garantiza la integridad del archivo para su instalación.

| Nombre                                                                                                 | Fecha de modificación | Tipo                | Tamaño    |
|--------------------------------------------------------------------------------------------------------|-----------------------|---------------------|-----------|
|  baselineProfiles     | 20/05/2025 21:19      | Carpeta de archivos |           |
|  casino.apk           | 20/05/2025 21:19      | BlueStacks.Apk      | 91.277 KB |
|  output-metadata.json | 20/05/2025 21:19      | Archivo JSON        | 1 KB      |

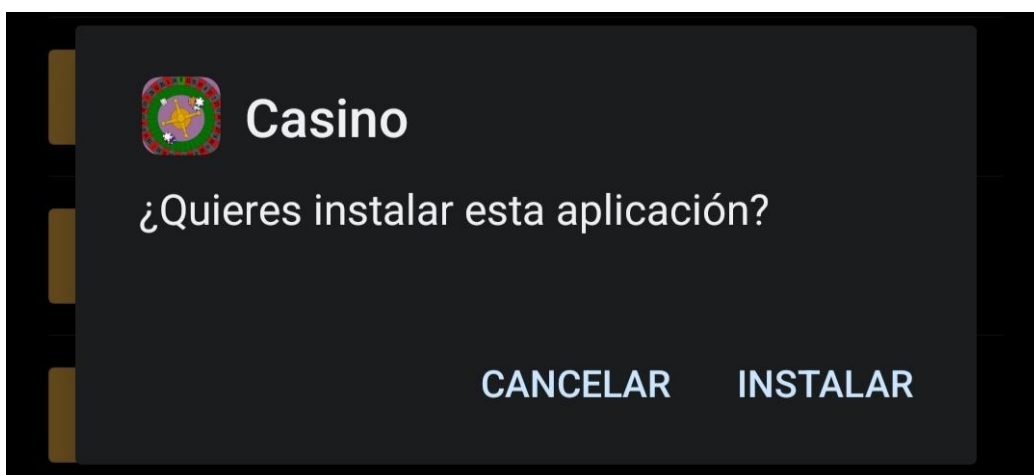
Actualmente, la distribución será gestionada directamente por el propietario de la aplicación, ya que aún no está disponible en Google Play ni en otras plataformas oficiales de descarga. Por ello, el APK podrá enviarse a través de métodos como Bluetooth, Gmail, USB o cualquier otro medio de transferencia compatible.

## Procedimiento de instalación

1. **Recibir el archivo APK** a través del método seleccionado (Bluetooth, correo, USB, etc.).
2. **Abrir el gestor de archivos** en el dispositivo móvil y buscar el archivo APK recibido.



3. **Presionar sobre el archivo APK** para que se abra un diálogo de instalación. Si ya existe una versión anterior instalada, la opción será actualizar.



4. Una vez finalizada la instalación, ir a la pantalla de inicio del móvil, localizar la aplicación y **abrirla para comenzar a usarla.**



## 10. Conclusión

### 10.1 Valoración general del proyecto

El desarrollo de esta aplicación ha representado un reto técnico y organizativo que ha permitido poner en práctica diversas competencias relacionadas con la programación móvil, el diseño de interfaces, el desarrollo backend, la gestión de bases de datos y la implementación de protocolos de seguridad. Desde la conceptualización hasta la puesta en marcha, se ha llevado a cabo un proceso completo que abarca tanto la planificación inicial como la solución de problemas técnicos y la optimización del rendimiento.

La aplicación cumple con los objetivos planteados inicialmente: ofrecer una experiencia entretenida, accesible y sencilla para un público adulto, a través de juegos casuales que fomentan momentos de ocio en espacios cortos de tiempo. La elección tecnológica ha sido acertada, combinando Kotlin con Jetpack Compose para un desarrollo ágil y moderno en Android, Java para un backend robusto mediante Jakarta EE, y una arquitectura de bases de datos híbrida que aprovecha lo mejor de MySQL y MongoDB para manejar datos de forma eficiente.

El sistema de autenticación basado en JWT garantiza la seguridad de las sesiones y la protección de los datos personales de los usuarios, y la interfaz gráfica, diseñada con cuidado, facilita la navegación y el uso, incluso para personas sin experiencia previa en juegos digitales.

## 10.2 Dificultades encontradas y soluciones implementadas

Durante el desarrollo, se presentaron diversas dificultades técnicas, entre ellas la gestión adecuada de la comunicación con el backend, la correcta serialización de objetos complejos, y la implementación de mecanismos para evitar bloqueos en la interfaz debido a la ausencia de conexión o la caída del API REST.

Uno de los principales desafíos fue asegurar que la aplicación no se cerrara inesperadamente cuando el servidor no estuviera disponible. Esto se solucionó mediante la implementación de interceptores con manejo de excepciones y respuestas controladas que permitieron que la aplicación notificara al usuario sin colapsar.

Asimismo, la integración de bases de datos relacionales y NoSQL supuso un esfuerzo para diseñar un sistema eficiente que aproveche la velocidad de MongoDB para operaciones temporales y la persistencia y robustez de MySQL para registros históricos, evitando así cuellos de botella y asegurando una experiencia fluida para el usuario.

## 10.3 Funcionalidades pendientes y posibles ampliaciones

El proyecto actual ofrece un MVP (Producto Mínimo Viable) que cubre las funcionalidades básicas de acceso, autenticación y juegos en Blackjack y Ruleta. Sin embargo, existen varias funcionalidades importantes que quedaron pendientes o se encuentran en fase prototipo y que se prevé integrar en versiones futuras para mejorar la experiencia general:

- **Integración completa del juego de tragamonedas:** Actualmente está en fase inicial o no disponible, y su implementación ofrecerá mayor variedad y atractivo a la oferta de juegos.
- **Opción para cerrar sesión desde los juegos:** Facilitará la gestión de usuarios y mejorará la usabilidad al permitir salir de manera rápida y sencilla sin necesidad de cerrar la aplicación.
- **Sistema de recargas y transacciones económicas:** Fundamental para simular un entorno de casino más realista, donde los usuarios puedan gestionar su saldo y realizar apuestas con dinero virtual o real.
- **Eliminación de cuentas:** Ofrecer esta opción contribuirá a cumplir con las normativas de protección de datos y a dar mayor control al usuario sobre su información personal.

## 10.4 Futuras mejoras técnicas y de usabilidad

Además de las funcionalidades antes mencionadas, se identifican varias áreas de mejora tanto en la parte técnica como en la experiencia de usuario:

- **Optimización del rendimiento:** Implementar técnicas adicionales para mejorar la eficiencia de las consultas y reducir la latencia, especialmente en situaciones de alta concurrencia.
- **Notificaciones y alertas:** Añadir notificaciones push para informar a los usuarios sobre eventos relevantes, promociones o recordatorios.
- **Ampliación de juegos y funcionalidades:** Incorporar nuevos juegos de casino, mini-juegos y opciones de personalización para atraer a un público más amplio.
- **Accesibilidad:** Mejorar la accesibilidad de la aplicación para personas con discapacidades, ajustando el diseño y la interacción.
- **Soporte multilingüe:** Facilitar el uso de la aplicación a usuarios de diferentes regiones mediante traducciones y ajustes culturales.

## 10.5 Reflexión final

En resumen, el proyecto ha sentado una base sólida para una aplicación de ocio digital que combina tecnología moderna y un diseño orientado al usuario. Aunque se han presentado limitaciones y funcionalidades aún por implementar, el trabajo realizado abre la puerta a futuras expansiones que pueden convertir esta aplicación en una plataforma completa de entretenimiento para dispositivos móviles.

La experiencia acumulada en este proceso ha sido valiosa, aportando no solo conocimientos técnicos sino también habilidades para planificar, organizar y afrontar los retos que surgen en el desarrollo de software. Se espera que, con las mejoras y ampliaciones previstas, el producto final cumpla con las expectativas de los usuarios y se convierta en una opción atractiva dentro del mercado de aplicaciones de juegos casuales.



## **11. Bibliografía**

Apuntes adquiridos en el curso

Página oficial de android

Stackoverflow y Stackoverflow en español

LottieFiles

Splash scree y lottie

Generador de imágenes y ayuda: ChatGPT

Página de las imágenes del mazo de cartas: Link

Discuss kotlin foro

Maven repositorios

Reddit

Github